Tampere University

Kaan Pide

COMPARING PERFORMANCES OF MINIMAX AND ALPHA-BETA SEARCH ALGORITHMS FOR A CHESS ENGINE

Bachelor's Thesis
Faculty of Engineering and Natural Sciences

Examiner: Zheying Zhang
April 2026

Acknowledging the use of artificial intelligence (AI) in student assignments

No I have not used generative AI in my assignment.

Yes I have used generative AI in my assignment. The details are provided below.

ABSTRACT

Kaan Pide: Comparing Performances of Minimax and Alpha-Beta Search Algorithms for a Chess Engine
Tampere University
Computing and Electrical Engineering
April 2026

This paper aims to compare the runtime performances of minimax and alpha-beta pruning algorithms for a chess engine. To compare these algorithms, initially a CLI chess game is created. Then the algorithms are implemented, and ways to accelerate their runtimes are investigated. To accelerate the minimax algorithm, multithreading with OpenMP API is used, and to accelerate the alpha-beta pruning algorithm, randomisation and multithreading is used. These algorithms are then tested with two sets of tests, measuring the runtimes from the initial position of a chess match, as well as a preset position that is more complex to analyse. The results indicate that with all proposed acceleration methods, up to 340 times faster runtime performances can be achieved.

Keywords: minimax, alpha-beta pruning, multithreading, acceleration

The originality of this thesis has been checked using the Turnitin OriginalityCheck service.

CONTENTS

1. INTRODUCTION 4
2. BACKGROUND 5
3. METHODOLOGY 6
    3.1 Implementation of the chess game 6
    3.2 Implementation of the algorithms and the evaluator 6
4. MINIMAX ALGORITHM 9
    4.1 Historical and mathematical overview 9
    4.2 Complexity and acceleration of the minimax algorithm 10
5. ALPHA-BETA PRUNING ALGORITHM 12
    5.1 Overview of the alpha-beta pruning algorithm 12
    5.2 Acceleration of the alpha-beta pruning algorithm 13
6. TESTING THE ALGORITHMS 14
7. COMPARING THE RESULTS 15
    7.1 Results of the repeated tests 15
    7.2 Results of the preset position tests 16
    7.3 Discussion 17
    7.4 Limitations 18
8. CONCLUSION 19
REFERENCES 20
APPENDIX: THE SOURCE CODE 21

LIST OF SYMBOLS AND ABBREVIATIONS

API	Application Programming Interface
CLI	Command Line Interface
GUI	Graphical User Interface
C++	The programming language used to develop the algorithms
CMake	The build system that manages the building, compilation and testing of the algorithms
OpenMP	The library that handles multi-platform shared-memory parallel programming for C/C++ and Fortran programming languages
GitHub	Distributed version control platform

1. INTRODUCTION

When we think of a strategy game, the first game that comes to mind is chess. Chess is over 1500 years old and ever since its invention, people always tried to find newer and better strategies to beat their opponents. This search accelerated with the invention of programmable computers, with them being able to process thousands or even millions of moves per second. Naturally, the search in computer science field then became on how to make the best and fastest chess engine possible.

The purpose of this research is to first create a barebones chess game on C++, and then to implement the engine algorithms. By the end of this research, there should be a playable chess game, alongside with an opportunity to play against a chess engine, which is chosen from the best performing algorithm.

The problem that this paper addresses is to compare the minimax [3] and alpha-beta pruning algorithms [7] in terms of their runtime efficiencies, and to explore possible steps to make the searching algorithms more efficient by utilizing parallel programming. The paper will also address briefly on making a chess game from scratch in C++. Therefore, this paper will aim to answer the questions:

1. How to build a chess game, as well as an engine to accompany it?
2. With what methods can the minimax and the alpha-beta pruning algorithm be accelerated?

The research process consists of firstly some background and literature reviews. After that, the implementation of the chess game is explored. Then, the algorithms are implemented and ways to accelerate those algorithms are researched. To test these algorithms, two sets of tests are prepared and tested across multiple parameters. Finally, the findings are analysed and drawn conclusions between the findings and the theory.

2. BACKGROUND

The idea of a man-made machine playing chess goes all the way back to the year 1770, with the first ever made chess engine “The Mechanical Turk” by Wolfgang Von Kempelen, however it cannot be truly classified as an engine, since it wasn’t an autonomous machine but rather a chess grandmaster controlled the Turk’s arm using a small mechanism from inside the machine. The grandmaster would see which piece has been moved from the small magnets underneath the board and replicate the move on his smaller chess board. He would then make the move by moving the Turk’s arm, essentially creating the illusion of the first autonomous chess engine [1] [9].

The first fully autonomous chess engine, however, did not come to life until 1914, with the introduction of El Ajedrecista by Torres y Quevedo [8] [9]. It had the capacity to checkmate the opponent’s only black king with its white king and a rook. This invention marked a new milestone within the chess community, and the beginning of a new era, but no more new inventions would have been made until the 1950s. Claude Shannon proposed the idea of a computer playing chess in his paper [2], where the main idea was to show how a computer can theoretically play chess with a set of predetermined instructions to evaluate the board on each turn, and make a move based on those instructions.

Shannon explained that a machine can play chess, theoretically albeit slow and ineffectively, using the minimax algorithm, in which the machine evaluates every possible move for White and Black, and tries to minimise the evaluation for Black’s turn and maximise the evaluation for White’s turn. However, such a machine would be slow and weak, as iterating through all possible moves in a chess game increases the complexity exponentially, in the order of 10^3 per move made by White and followed by Black. It takes about 40 moves on average for a chess game to end, thus there would be about 10^{120} different variations of a chess game from the initial position. It may be theoretically possible to calculate all of those moves, but even with a machine that can calculate one move per microsecond, it would still take over 10^{90} years to finish calculating just the starting move [2].

3. METHODOLOGY

3.1 Implementation of the chess game

Before starting to evaluate and compare the chess algorithms, first, the game of chess itself is needed. The game is coded in the programming language C++ and contains some of the most important parts of chess, but not all of them. Namely, there exists in game the ability to move a pawn from its starting position two squares, the ability to castle the king if the king and the rook haven't made a move yet, and the pawn that reaches the other side of the board to become a queen. There isn't the ability to check the king, and by extension, pinning a piece to the king doesn't stop its ability to move. The checkmate logic is when the king is captured, and not when the king has no safe space to move and is in check, however this change in logic does not affect the ultimate outcome of the game, as a position in checkmate leads to the capture of the king on the next move. The special rule of chess "en passant" is also not coded in.

The game can be played in a CLI, for its simplicity at the cost of the visuals that a GUI can bring. Before starting the game, the CLI prompts the player to enter their desired colour to play with, alongside the depth of the engine. The main loop of the game then starts, with the White's move, if it is the player who has the white pieces, then the game prompts the player to enter a move, which is the starting square of the desired piece, followed by a whitespace and the ending square in chess coordinates. If the engine starts first however, it'll run the first move and display the board with its move already played, then prompts the player to enter their move. This goes on and on until either the player quits the game, or either side is checkmated.

3.2 Implementation of the algorithms and the evaluator

The engines take the chess board and its associated parameters and perform their calculations accordingly to determine the best move of the position. To achieve this, the engines run a two-part algorithm. The main part of the algorithm is running the algorithm itself through all possible moves, or the moves the engine deems worthy of calculation. This is the main reason why the algorithms grow exponentially with the increased depth, as the complexity of chess grows in three-orders of magnitude per two moves made (one for White and one for Black). The second part of the algorithm is run when the algorithm reaches its deepest depth, where it evaluates the board to see which colour has the advantage and by how much. Fundamentally, this evaluation function determines which

move is ultimately made, since the main part of the algorithm is deterministic and will choose the same move every time for a given board. Both algorithms and their domains are shown in Fig. 1:

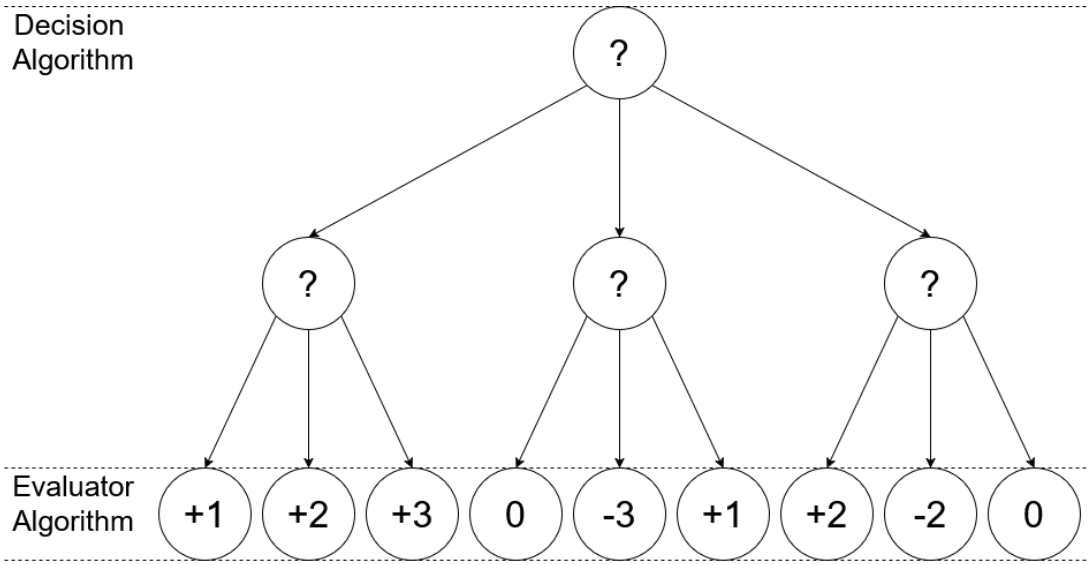


Figure 1: Each node represents a unique game state, and each edge represents the move to get to that game state from the parent game state. Decision algorithm aims to find the values of the intermediate nodes, while the evaluator algorithm aims to get the game value at the lowest depth.

For this reason, the chosen evaluation function is rather simplistic and open for improvement. It evaluates the board based on three factors: the total weighted piece count, their centrality and the total weighted available moves. The function uses whole numbers to calculate the value of the board, since whole numbers are more efficient in calculating compared to floating-point numbers. For each piece, they are added to the total “evaluation score”. This depends on two factors, the type of piece and its colour. The type of piece determines its weight on the board, as such a pawn is worth only 10 points, whereas the queen has 90 points [3]. If the piece is white, it is added to the evaluation score and subtracted if it is a black piece. This results in the score being a positive number if the White has the advantage, and conversely a negative number if Black has the advantage.

The centrality of pieces measures how effective they are within the board, and this measure changes based on the piece. For the pawns, this value increases as they advance through the board, since they are closer to promotion. For the kings, they have the maximum centrality in their castled position, with the bias being towards their side of the board. The remaining pieces have a basic measure, which is a monotonic increase as the piece approaches the centre of the board.

The total weighted available moves count the total number of moves a given colour can make on the board. For all the pieces, their possible moves are listed and added to the evaluation based on their arrival squares, such that if a piece controls a central square, that adds more to the evaluation score, but a piece that only controls the edge squares contribute less overall. This aims to ensure that the engine favours a move that have more control over the board.

Measuring the total time elapsed for one engine move is done using the “chrono.h” library of the C++ language. Before running the algorithm, the library records the current time in the order of smallest possible time tick for the given device. The algorithm is run as soon as possible to make sure no time is wasted between time measures. When the algorithm finishes calculating, the library gets the current time with the same function and precision. The difference between the starting and the ending time is then calculated and converted into the order of milliseconds. Finally, the total time elapsed for running the algorithm is displayed on the CLI.

4. MINIMAX ALGORITHM

4.1 Historical and mathematical overview

The term minimax is a portmanteau of the words “minimum” and “maximum”, since the main idea of the algorithm is to maximise minimum gains. It is also sometimes called the “maximin” for this reason. The algorithm was first proposed by John von Neumann in 1928 [4]. The paper was the solution to the question “n players $S_1, S_2, S_3, \dots S_n$ are playing a given strategy game G. How must one of the participants, S_m, play in order to achieve a most advantageous result?” [4].

For each step of the game G, the players must decide on a strategy $x_1, x_2, \dots x_n$, after which they receive their rewards based on all the strategies played on that step as $g(x_1, x_2, \dots x_n)$. In the case of chess, $n = 2$ and the players S_1 and S_2 choose their strategies x and y, respectively. This results in the game value being $g(x, y)$. The players then would want to have the game value in their favour, for S_1 by maximising and for S_2 by minimising. Since the players’ choices are independent, it can be assumed that the player S_2 will always choose the strategy y that returns the minimum value for $g(x, y)$, which is $\min_y g(x, y)$. To get the most advantageous result for S_1, the value must be maximised with respect to the strategy x, which gives the equation $\max_x \min_y g(x, y)$. As the player S_1 cannot influence the strategy y chosen by player S_2, the value of $\max_x \min_y g(x, y)$ can only grow larger with suboptimal strategy choices by S_2, thus giving a lower bound for the equation:

$$\geq \max_x \min_y g(x, y)$$

This holds true for S_2, with an upper bound:

$$\leq \min_y \max_x g(x, y)$$

For the engine algorithm to make the best moves possible, it must assume both players S_1 and S_2 plays perfectly, resulting in a minimising and maximising loop on each move. The best move is determined by running the algorithm recursively until the desired depth is reached, when the evaluator function returns the game value $g(x, y)$. Then on each step going backwards from the call stack, each player chooses the best move among all the possible moves that minimises or maximises the values depending on their strategy [4]. A simplified example of this can be seen in Fig. 2:

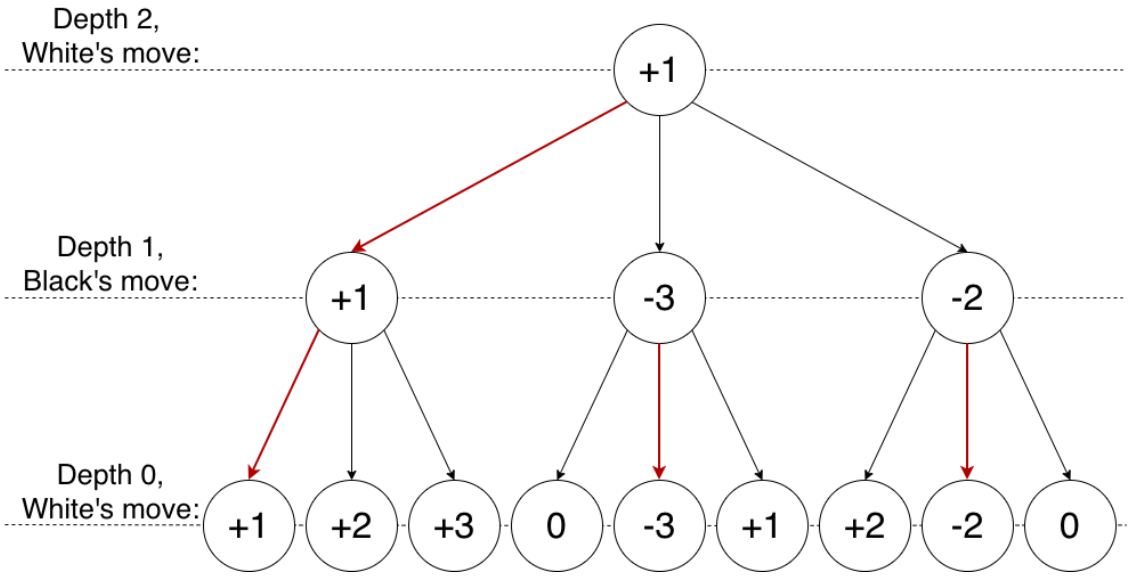


Figure 2: Simplified decision stack of a minimax algorithm. Each node represents a game state, with the game values inside them, each edge corresponds to a possible move from that game state and each move chosen by the players with respect to the minimax algorithm are highlighted with red arrows.

In Figure 2, the evaluation function yields the values for the depth 0. Going back the stack, Black chooses the minimum values among each move, the values +1, -3 and -2 respectively. On the highest depth 2, White then chooses the move which maximises the value, which were minimised by Black. Overall, the result of White's chosen move becomes the move that yields +1 evaluation. However, it should be noted that +1 is only a lower bound, as after White's move, any differing move of Black's only increases the value from +1 up to +3. This decision mechanism makes this algorithm so powerful, but ultimately it is bounded in strength by the evaluation function $g(x, y)$.

4.2 Complexity and acceleration of the minimax algorithm

In theory, given a perfect evaluator and infinite processing power, every branch of a chess game can be analysed and compared. This would make the engine unbeatable, as it calculates all the possibilities. Achieving this would be near-impossible, if not completely impossible, given the exponential nature of chess, increasing the number of possible moves by 3 orders of magnitude in every 2 moves (White's move followed by Black's). For a 3-move analysis in the beginning of the game, or a depth 6, there are more than 300 million possible moves to analyse, which is the main drawback of this algorithm.

One of the ways to accelerate this algorithm is the use of multithreading, by giving each branch of the highest depth to its own thread. According to Amdahl's law [5], total runtime of a program is determined by its serial and parallelizable parts:

$$T = W_{ser} + W_{par}$$

With the number of parallel resources P, the runtime can be lowered down to:

$$T_P \geq W_{ser} + \frac{W_{par}}{P}$$

For the minimax algorithm, the serial part of the algorithm is the comparison of branches, as each branch needs to be compared to the rest, and the parallelizable part is the lower depths, since no branch are dependent on any other branch in terms of calculations. Thus, it is satisfactory to distribute the workload on depth $d - 1$ to the available resources to run in parallel. The serial part can be negligible in the scheme of the whole algorithm, so the expected speed up for P parallel resources is:

$$T_P \geq W_{ser} + \frac{W_{par}}{P} \approx \frac{W_{par}}{P}$$

This multithreading is achieved via the OpenMP API for the C++ language. The workload of each branch at the highest depth is split into their own threads, and the subsequent calls are made to the sequential version of the algorithm, since the number of moves on the highest depth is almost always large enough to warrant the use of multithreading, while introducing parallelism to lower depths may be more costly due to the thread creation and destruction overhead. The comparisons between the sibling branches are done with critical sections to prevent race conditions [10].

5. ALPHA-BETA PRUNING ALGORITHM

5.1 Overview of the alpha-beta pruning algorithm

The alpha-beta pruning algorithm is an acceleration for the minimax algorithm. The main idea is the same, with the small difference of pruning suboptimal branches. This is achieved by keeping track of the local minimum and maximum game values in variables, commonly named alpha and beta, hence the algorithm's name. Initially the variables are initialized as worst possible outcomes:

$$\alpha = -\infty, \qquad \beta = +\infty$$

The worst possible evaluation for White is the minus infinity, and the plus infinity for Black. Then, at each depth, the values found are compared to the opposite colour's local minima or maxima. If the opponent already has a move that is more advantageous to the one calculated, then the rest of the moves can be skipped, as calculating those moves will result in them ultimately not being chosen by the algorithm. This allows to prune whole branches without evaluating them, and in especially deeper depths, this results in a considerable number of moves being pruned. A simplified example of this can be seen in Fig. 3:

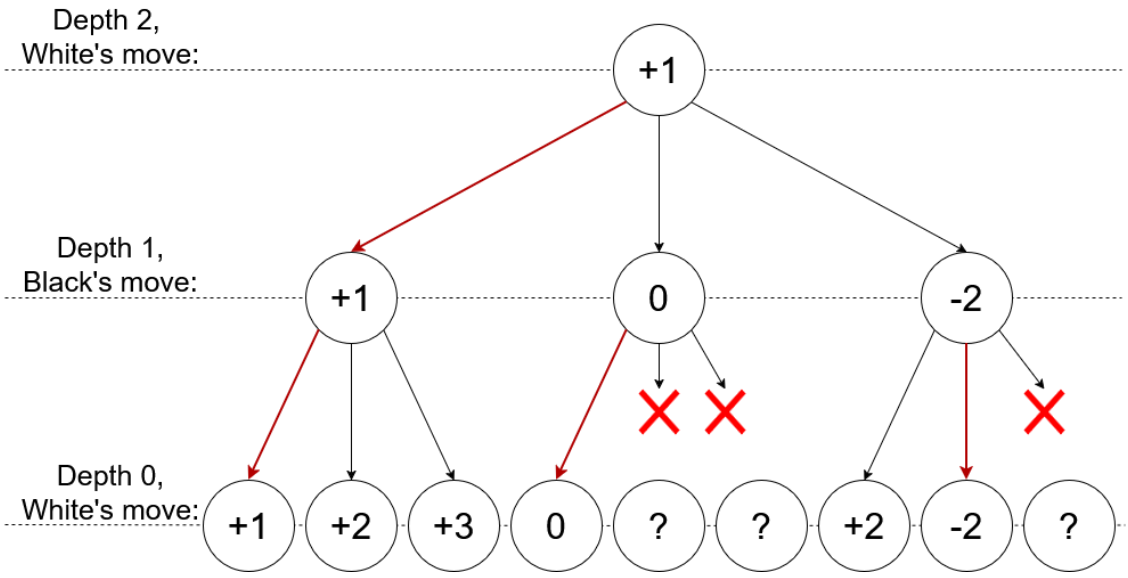


Figure 3: Simplified decision stack of an alpha-beta pruning algorithm. The branches that are determined to be suboptimal are pruned and shown by the red crosses.

In Figure 3, initially the first branch is evaluated for the depth 1, with the value +1, since it is the smallest game value among the available moves. Then the variable alpha, storing

White's best move, is updated to that value. In the second iteration, when reaching depth 0, the game value is found to be 0, which is less than the local maxima (alpha) of +1. This implies that no matter what the subsequent game values are, on depth 2 White will never choose this branch, thus rest of the moves are pruned and never calculated. Continuing with the last branch, first the value +2 is obtained. This value is larger than the alpha, thus the search for a better move for Black continues. When the value -2 is found, it is once again clear that on depth 2, White will never consider this path, thus the remaining of the branches are pruned.

5.2 Acceleration of the alpha-beta pruning algorithm

A small step that helps with acceleration of the alpha-beta algorithm is the introduction randomisation. Randomising the set of moves before running the algorithm aims to reduce bias and help finding better moves earlier, so more branches can be pruned, accelerating the algorithm.

Due to the nature of this algorithm, it isn't trivial to apply multithreading to its branches. Every branch needs to keep track of the alpha and beta variables, which are dependent on the sibling branches, and if they are given their own threads, those branches need to be frequently stopped execution to wait for another branch to update the alpha and beta variables. And if this functionality is not implemented correctly, it can introduce race conditions, resulting in branches that would be pruned being not pruned at all. Even if it is implemented correctly, branches waiting for others to update the variables may result in a larger overhead compared to running the algorithm sequentially.

To get around this issue, parallelism of alpha-beta pruning algorithm is done via running the minimax algorithm on parallel on the highest depth. In that depth, there will not be any pruning, since pruning a branch is dependent on the local minima or maxima. This allows the use of minimax algorithm on the highest depth only, as it will not impact the performance of the alpha-beta pruning algorithm. Furthermore, the minimax algorithm on the highest depth allows for multithreading, as explained in the previous chapter. With the parallelised minimax algorithm on the highest depth, calling the sequential alpha-beta pruning algorithm on each thread, it results in an algorithm that is both efficient in the moves it analyses and fast since it exploits parallel resources.

The alpha-beta pruning algorithm by itself can speed up the decision stack massively, as just finding one move that leads to a disadvantageous position result in a substantial number of moves not being calculated. While this is a significant advantage, the exponential nature of chess is still the main factor for the runtime of these algorithms.

6. TESTING THE ALGORITHMS

The algorithms to be tested are the sequential minimax algorithm, the parallelised minimax algorithm, the sequential alpha-beta pruning algorithm, and the parallelised alpha-beta pruning algorithm. These algorithms are tested in two ways, first being a depth performance test, in which all four algorithms participate. The second is calculating a move from a preset position, in which only the parallelised algorithms will participate.

In the first test, all four algorithms are initialised with the same depth, and given the same position to calculate. The position is the starting position of chess, and the algorithms will calculate the White's move. Since the nature of computing and multithreading can result in some fluctuations in the runtime, these algorithms are run repeatedly, and their mean and standard deviation is displayed.

For the second test, a preset position is used. This position is the 37th move of the second game between Garry Kasparov and IBM's Deep Blue in 1997 [6]. The match between Deep Blue and Kasparov ended in 3.5-2.5 in favour of Deep Blue, marking a huge milestone in the chess engine world. This game was chosen for its significance. To test, the board is given to both parallel algorithms, and not the sequential ones, as the outcome will be the same on both parallel and sequential cases. The reason for this test is to measure the runtime in a more complicated board with more moves available, as well as to draw a relative performance between the algorithms and the Deep Blue.

The tests are conducted on an Intel® Core™ i5-14600KF, 3500 MHz, 14 Core, 20 Threads CPU. While the tests are run, there were no other programs running. The tests cases are coded without any testing software, only by using the functionality provided by the language C++ and CMake build system. The standard for the language C++ is the ISO International Standard ISO/IEC 14882:2024(E) – Programming Language C++ (C++23). The program is compiled with the MinGW toolchain.

7. COMPARING THE RESULTS

7.1 Results of the repeated tests

The first test is conducted with three different depths, measuring all four of the algorithms. The depths and their repetitions are 1000 times for depth 4, 100 times for depth 5, and 10 times for depth 6. The number of repetitions decrease due to the exponential nature of the algorithms. The results of the runtimes are as follows:

	Minimax	Parallelised Minimax	Alpha-beta Pruning	Parallelised Alpha-beta Pruning
Depth 4, Repeat 1000	$\mu = 271.36\ ms$ $\sigma = 2.70\ ms$	$\mu = 28.64\ ms$ $\sigma = 1.56\ ms$	$\mu = 19.79\ ms$ $\sigma = 4.41\ ms$	$\mu = 5.30\ ms$ $\sigma = 0.73\ ms$
Depth 5, Repeat 100	$\mu = 6866.09\ ms$ $\sigma = 34.49\ ms$	$\mu = 868.71\ ms$ $\sigma = 22.06\ ms$	$\mu = 216.04\ ms$ $\sigma = 68.27\ ms$	$\mu = 75.12\ ms$ $\sigma = 8.22\ ms$
Depth 6, Repeat 10	$\mu = 170881.40\ ms$ $\sigma = 191.04\ ms$	$\mu = 20133.30\ ms$ $\sigma = 582.16\ ms$	$\mu = 2403.90\ ms$ $\sigma = 339.58\ ms$	$\mu = 515.00\ ms$ $\sigma = 34.07\ ms$

Table 1: The runtime results from repeat testing with depths 5 and 6

Across all the depths, the parallel versions are always faster than their sequential counterparts, as well as the alpha-beta pruning algorithm is always faster than minimax algorithm, regardless of their use of parallel resources. The relative speed ups between the minimax and the alpha-beta pruning algorithm aren't constant. For example, on depth 5, alpha-beta pruning algorithm is $\frac{6866.09}{216.04} = 31.78$ times faster, while on depth 6, the speed up is $\frac{170881.40}{2403.90} = 71.09$, more than twice the difference across the depths. This is due the nature of alpha-beta pruning, as the deeper the search tree, there are more opportunities to prune a branch, making the algorithm faster as the depth increases.

To compare the fluctuations in the measurements, the coefficient of variation is used. It is defined as the ratio of standard deviation to the mean:

$$CV = \frac{\sigma}{\mu}$$

Calculating the coefficients of variation, the following table is obtained:

	Minimax	Parallelised Minimax	Alpha-beta Pruning	Parallelised Alpha-beta Pruning
Depth 4, Repeat 1000	$CV = \frac{2.70}{271.36} = 0.010$	$CV = \frac{1.56}{28.64} = 0.054$	$CV = \frac{4.41}{19.79} = 0.223$	$CV = \frac{0.73}{5.30} = 0.138$
Depth 5, Repeat 100	$CV = \frac{34.49}{6866.09} = 0.005$	$CV = \frac{22.06}{868.71} = 0.025$	$CV = \frac{68.27}{216.04} = 0.316$	$CV = \frac{8.22}{75.12} = 0.109$
Depth 6, Repeat 10	$CV = \frac{191.40}{170881.40} = 0.001$	$CV = \frac{582.16}{20133.30} = 0.029$	$CV = \frac{339.58}{2403.90} = 0.141$	$CV = \frac{34.07}{515.00} = 0.066$

Table 2: The coefficients of variation for all algorithms

The coefficients are lowest on the sequential minimax algorithm, and largest on the sequential alpha-beta pruning algorithm. This implies that the minimax algorithm has a more stable and centred distribution of runtime samples compared to the alpha-beta pruning algorithm. This can be explained by the nature of both algorithms, minimax algorithm will always exhaust all possible moves before returning, while alpha-beta pruning algorithm will shuffle the moves before pruning the branches. The randomisation of moves allows alpha-beta pruning algorithm to find a better move faster, at the cost of stability on the runtime.

7.2 Results of the preset position tests

For the second test, only parallelised versions of minimax and alpha-beta pruning algorithm are used. The preset position is given to both algorithms with depths 4 and 6. On this test, the main goal isn't to find the fastest algorithm but rather find out the slowdown that a more complex position brings, therefore in this test, the algorithms are run only once. The results of the runtimes are as follows:

	Parallelised Minimax	Parallelised Alpha-beta Pruning
Depth 4	$218\ ms$	$66\ ms$
Depth 6	$331394\ ms$	$8937\ ms$

Table 3: The runtimes of the parallel algorithms that are given a preset position

Since the position isn't the starting position but rather a preset position, there are more possible moves to calculate at every step. This makes the algorithms slower as the game progresses. However, it should be noted that in the endgame, where most of the pieces are gone, the algorithms run faster due to the small number of possible moves.

In this preset position from 1997 between Kasparov and Deep Blue [6], the chess engine Deep Blue played the move bishop from c2 to e4, maintaining the advantage it had over the World Champion. Both algorithms, in both depth cases, preferred the move rook from a2 to a8.

To improve the gameplay accuracy of the algorithms, the evaluator should be improved, as the algorithms themselves are only a decision stack, choosing the best possible move among the moves. If a better evaluator function is used, the gameplay performance of the algorithms would improve.

7.3 Discussion

The results from both test cases reflect the added acceleration methods, as well as the added efficiency of the alpha-beta pruning algorithm. For example, on depth 6, the sequential minimax algorithm took nearly 3 minutes to execute, meanwhile the parallelised alpha-beta algorithm achieved the same result in less than one second. This amounts to about 340 times faster execution time for the algorithm.

The speed ups between the minimax and parallelised minimax are near constant across all depth, at about almost 8 times the speed up. This may be due to how multithreading works, since at each depth, the amount of work is split into the same number of threads, with the same running environment. The speed up from multithreading is therefore only constant and can never keep up with the exponential nature of chess.

The speed ups between parallelised minimax and sequential alpha-beta pruning, however, is not constant across all depths. For depth 4, it is about twice as faster, and as the depth grows larger, the difference only grows alongside it. The reason for this difference may be caused by the pruning algorithm getting more efficient as the depths

grow, as more branches means more opportunities to prune a branch. The minimax algorithm will exhaust all branches, parallelised or not, while alpha-beta pruning algorithm tries to choose a branch that shows promise, and discard the ones that are not beneficial for the game. This difference allows the sequential alpha-beta pruning algorithm to surpass even the parallelised minimax algorithm.

The speed ups between the alpha-beta pruning algorithm and its parallelised counterpart are also near constant across the depths, being about 3-5 times faster. This speed up is less than the speed up for the minimax and its parallelised counterpart, and this difference may be caused by how the parallelism is achieved on alpha-beta pruning algorithm. Since the algorithm is dependent on the local minima/maxima, the way to parallelise is to use minimax on the highest depth, parallelise that depth, and run sequential alpha-beta pruning algorithm from those branches. This, however, skips the pruning step of one depth, thus not benefitting as much from parallelism as minimax does.

Finally, the difference in runtime between the initial position and a more complex position, measured in the second test, is about 10-15 times slower for all depths on both algorithms. This is due to the increased number of moves per piece on the board, since in the initial position, not many pieces can move around. Increasing the available moves on the board also increases the subsequent available moves from each of those moves, increasing the total number exponentially.

7.4 Limitations

The limitations of this study are primarily the inefficiency of both algorithms and the low number of algorithms analysed. Since both algorithms are exhaustive, the exponential nature of chess will always make these algorithms suboptimal unless a cleverer way is used to prune the branches. For alpha-beta pruning algorithm, the pruning part can only happen after the board is analysed, which is at the lowest depth. This leads to the algorithm on higher depths; the pruning of a bad move can only happen after analysing a significant part of that branch. If a move can be predetermined to be not beneficial, a considerable amount of time can be cut calculating suboptimal moves.

That can be implemented with a strong evaluator function, and if a move made is over a certain threshold worse compared to its sibling branches, it can be considered a bad move without checking the rest of its branch, leading to a more robust pruning system. This, however, would be a non-exhaustive way of searching the game tree, and may miss a good move in the long run which is worse on the short term.

8. CONCLUSION

This paper explored the how to make a chess engine and ways to accelerate the algorithms' runtime performances. Initially, a barebones CLI chess game was implemented, and after that, the minimax and the alpha-beta pruning algorithms were added. Those algorithms are then accelerated with methods such as multithreading and randomisation. Two tests were set and the runtimes of the algorithms across multiple depths were measured. From the results of the tests, the peak speed up of 340 times faster going from the sequential minimax to parallelised alpha-beta pruning algorithm can be drawn.

For the future research, the percentage of moves pruned by the alpha-beta pruning algorithm can be explored, since the deeper the algorithm gets, the more it will prune. The exact percentages for a given depth might prove useful in terms of gaining insight on the efficiency of the algorithm. These percentages can give valuable information if the algorithm is to be improved as explained in the limitations subchapter, by introducing a more robust and non-exhaustive searching algorithm that can prune a branch if it seems to be a suboptimal move without exploring it in all its entirety.

Another topic to consider can be implementation of different evaluator functions, as the gameplay strength of the engine depends on the evaluator function; the decision algorithms have no real impact on the outcome. For this research, a lightweight and simple, three-way evaluator function was used, and the impact of a more heavyweight and thorough evaluator function can be tested to find out how costly it would be, while achieving better gameplay performance.

As discussed in the limitations subchapter, another algorithm that has a better pruning rate and logic compared to alpha-beta pruning algorithm can boost the runtime of the algorithm considerably, and a research on this improved alpha-beta algorithm can uncover the ways to achieve this, compare its runtime performance against the minimax and alpha-beta pruning algorithms, and also to test if the added pruning logic of removing a bad looking move without thoroughly checking its branch can still achieve the same level of gameplay performance as the more exhaustive approaches of both minimax and alpha-beta pruning algorithms.

REFERENCES

[1] Stephens, Elizabeth. “The Mechanical Turk: A Short History of ‘Artificial Artificial Intelligence.’” *Cultural Studies (London, England)* [ABINGDON], vol. 37, no. 1, January 2023, pp. 65–87, https://doi.org/10.1080/09502386.2022.2042580.

[2] Shannon, Claude E. “XXII. Programming a Computer for Playing Chess.” *The London, Edinburgh and Dublin Philosophical Magazine and Journal of Science*, vol. 41, no. 314, March 1950, pp. 256–75, https://doi.org/10.1080/14786445008521796.

[3] Madake, Jyoti, et al. “CHESS AI: Machine Learning and Minimax Based Chess Engine.” *2023 International Conference for Advancement in Technology (ICONAT)*, 2023, pp. 1–6, https://doi.org/10.1109/ICONAT57137.2023.10080746.

[4] Neumann, John von. “ON THE THEORY OF GAMES OF STRATEGY.” Contributions to the Theory of Games (AM-40), Volume IV, edited by Albert William Tucker and Robert Duncan Luce, vol. 4, Princeton University Press, 2016, pp. 13–42, https://doi.org/10.1515/9781400882168-003.

[5] Amdahl, Gene M. “Computer Architecture and Amdahl’s Law.” *0b00006481cf6df7* [LOS ALAMITOS], vol. 46, no. 12, no. 6689270, December 2013, https://doi.org/10.1109/MC.2013.418.

[6] Campbell, Murray, et al. “Deep Blue.” *Artificial Intelligence* [AMSTERDAM], vol. 134, no. 1, 2002, pp. 57–83, https://doi.org/10.1016/S0004-3702(01)00129-1.

[7] Abdelbar, Ashraf M. “Alpha-Beta Pruning and Althöfer’s Pathology-Free Negamax Algorithm.” *Algorithms* [Basel], vol. 5, no. 4, December 2012, pp. 521–28, https://doi.org/10.3390/a5040521.

[8] McShane, Luke. “El Ajedrecista.” *The Spectator (London. 1828)*, vol. 349, no. 10111, June 11, 2022, p. 58.

[9] Monnens, Devin. “‘I Commenced an Examination of a Game Called “Tit-Tat-to”’: Charles Babbage and the ‘First’ Computer Game.” *DiGRA 2013 - Proceedings of the 2013 DiGRA International Conference: DeFragging Game Studies*, 2013.

[10] Wang, Yu, et al. “Automatic Detection, Validation, and Repair of Race Conditions in Interrupt-Driven Embedded Software.” *IEEE Transactions on Software Engineering* [LOS ALAMITOS], vol. 48, no. 1, January 2022, pp. 346–63, https://doi.org/10.1109/TSE.2020.2989171.

APPENDIX: THE SOURCE CODE

Following algorithm in Program 1 is the sequential minimax algorithm. The rest of the algorithms, alongside with the evaluator function, the implementation of the chess game, a basic overview of how to operate the program, and both conducted tests can be found in the following GitHub page: https://github.com/klsaerf/Chess-Thesis

```
1  std::pair<Move, int> Engine::minimax(const ChessBoard &chessBoard,
   const int depth, const Color color) {
2     // If the game is over, return without checking deeper
3     const Color gameOver =
   ChessBoardFunctions::isGameOver(chessBoard);
4     if (gameOver != EMPTY) {
5         // Add depth information to a winning move, so a faster
   checkmate is preferred
6         const int winEval = gameOver == WHITE ? WHITE_WIN + depth :
   BLACK_WIN - depth;
7         return {NO_MOVE, winEval};
8     }
9     // Base recursion step
10    if (depth == 0) {
11        return {NO_MOVE, Evaluator::evaluate(chessBoard)};
12    }
13
14    // Tracks the best move for the current depth
15    std::pair<Move, int> bestMove;
16
17    // The moves to be made into the next depth
18    std::vector<Move> moves;
19    ChessBoardFunctions::getAvailableMoves(chessBoard, moves, color);
20
21    // White color base
22    if (color == WHITE) {
23        // Initialize with worst move for white -> black win
24        bestMove = {NO_MOVE, BLACK_WIN};
25
26        for (Move move : moves) {
27            // Copy of the board as to not mess with other moves
28            ChessBoard previous = chessBoard;
29            ChessBoardFunctions::makeMove(previous, move, color);
30            const std::pair<Move, int> newBest = {move,
   minimax(previous, depth - 1, BLACK).second};
31
32             // If white's move is better than the current best,
   update it
33            if (newBest.second > bestMove.second) bestMove = newBest;
34        }
35
36    } else if (color == BLACK) {
37        // Initialize with worst move for black -> white win
38        bestMove = {NO_MOVE, WHITE_WIN};
39
40        for (Move move : moves) {
41            // Copy of the board as to not mess with other moves
```

```
42            ChessBoard previous = chessBoard;
43            ChessBoardFunctions::makeMove(previous, move, color);
44            const std::pair<Move, int> newBest = {move,
45  minimax(previous, depth - 1, WHITE).second};

46            // If black's move is better than the current best, update
    it
47            if (newBest.second < bestMove.second) bestMove = newBest;
48          }
        }
49
50        return bestMove;
51  }
```

Program 1. *Sequential minimax algorithm implementation in engine.cpp, lines 9-60.*